CMPSC 448 Midterm Project

Cristian Aquino

CristianAquino22/Midterm-LLM-Classifier: identifying which LLM wrote a response using CNN and LSTM

Problem and dataset

The goal of this project is to figure out which LLM wrote a given AI chatbot response. I did a 3 class text classification problem. Given an AI generated response, predict whether it came from Claude, GPT, or Llama. I used the LMSYS Chatbot Arena Conversations dataset on the Hugging website (lmsys/chatbot_arena_conversations). This dataset contains 33,000 real user conversations with 20 different models. This dataset was built by taking each user prompt and sending it to two randomly paired models, and recording both answers. The dataset is gated, so I accepted its terms of use, and then in my notebook downloaded it and rebuilt it.
I picked three families from three different AI companies: GPT (gpt-3.5-turbo, gpt-4), Claude (claude-v1, claude-instant-v1), and Llama (llama-13b). I left out models such as Vicuna and Koala, as they were trained on ChatGPT conversations. When cleaning the dataset, I kept only English, single-turn conversations, so every example is one prompt and one response. I removed conversations flagged by the dataset's built in OpenAI moderation labels to avoid harmful content, and I also dropped empty responses and exact duplicates. After doing this, I had 6,958 GPT, 5,151 Claude, and 1,581 Llama responses. I randomly sampled each family to 1,581, for 4,743 examples total. I split the data 70/15/15 into a training set, which 3316 responses, validation set, with 719, and a testing set with 708. The split is grouped by prompt: since each prompt was sent to two models, a random split could put the same prompt in both train and test, and the model could learn to recognize prompts instead of writing style. Grouping makes sure each prompt shows up in only one split.

Algorithms and training

I tokenized text with a regular expression that keeps words, punctuation, and newlines as separate tokens, and I did not lowercase anything. I made that choice on purpose, since formatting and capitalization are probably part of each model's own format which helps the algorithms differentiate. The vocabulary was built from the training set only (top 20,000 tokens appearing at least twice), and unknown tokens map to <unk>. Sequences are truncated to 400 tokens. For the input+output setting, I used the first 100 tokens of the prompt, a [SEP] token, and the first 300 tokens of the response.

CNN

The CNN I built works as a 128-dimensional word embedding (trained from scratch), followed by three parallel 1D convolutions with filter widths 3, 4, and 5 (100 filters each), ReLU, global max pooling, dropout of 0.5, and a linear layer to 3 classes. Each filter acts like a learned n-gram detector, so the model picks up short phrases and patterns wherever they appear in the text.

LTSM

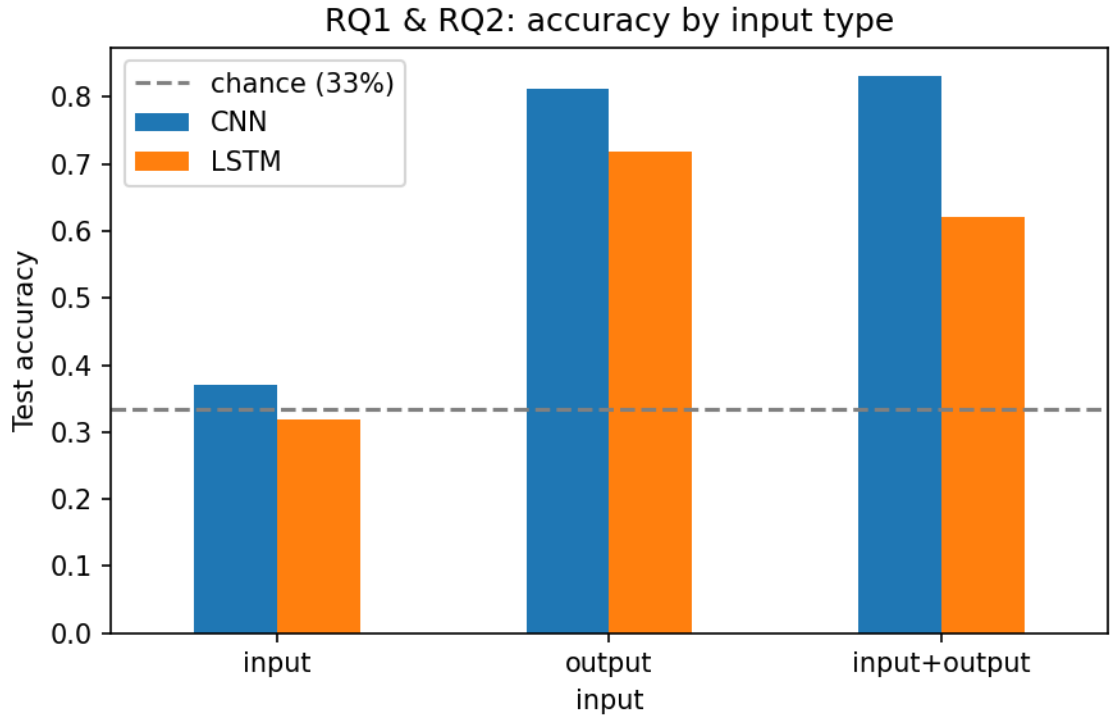
I built a bidirectional LSTM that uses the same 128-dimensional embedding, an LSTM with 128 hidden units per direction, packed sequences so padding is ignored, and the final forward and backward hidden states concatenated. Then dropout of 0.5 and a linear layer to 3 classes.

Both models used cross-entropy loss, the Adam optimizer (learning rate 0.001), and batch size 64, for up to 10 epochs. I used early stopping on validation macro-F1 with a patience of 2 epochs and restored the best checkpoint before testing. I trained on a free T4 GPU in Google Colab with random seed 22.

Results

I decided to evaluate on the testing set (708 examples) using accuracy and macro-F1. With three balanced classes, random guessing and a baseline gets about 33%.

	model	input	test_acc	test_macro_f1
0	CNN	output	0.812	0.813
1	LSTM	output	0.719	0.717
2	CNN	input	0.370	0.361
3	LSTM	input	0.318	0.306
4	CNN	input+output	0.832	0.833
5	LSTM	input+output	0.620	0.620



Can we identify which LLM generated a response?

Yes. Using only the response, the CNN reached 81.2% accuracy and the LSTM reached 71.9%, both far above the 33% baseline. This indicates that models do leave their own footprint in their writing.

Does the user's prompt help?

No. Using the prompt alone, both models were performing at a chance rate (37.0% and 31.8%). Adding the prompt to the response gave the CNN a small bump (81.2% to 83.2%), but the LSTM got worse (71.9% to 62.0%). The signal is in the AI's response, not the prompt given by the user.

Analysis

The fact that input only fails does not indicate any error. There's no real link between what the user asked and which model answered. If input-only had scored well, that would have been a red

flag for data leakage. Getting chance is actually evidence that the prompt-grouped split worked. In a different dataset this could change. For example, if certain models were mostly used for coding, the prompt topic alone could give the model away.

There are a few reasons of why CNN beats LSTM. First, the LSTM overfit. By its last epoch on output-only data, its training loss was down to 0.055 while validation F1 stalled around 0.70, so it was memorizing the training set. The CNN's training loss stayed around 0.4–0.5 and it generalized better. Second, each model's fingerprint seems to be local, which means the use of specific phrases, punctuation habits, and formatting. This indicates that CNN doesn't need the long-range memory an LSTM provides.

Adding the prompt cuts the response down to 300 tokens and adds text that carries no signal (as shown in the second answered question in the Results section). The CNN can ignore that noise because max pooling only keeps the strongest features. The LSTM has to read the whole sequence in order, so the noise gets mixed into its final hidden state. Llama was easiest to spot, which makes sense because after some research, llama-13b is a base model rather than a chat-tuned one, so its responses read different than the others. GPT was the hardest one. The biggest single error was GPT being predicted as Llama (42 cases). My hypothesis is that short or plain GPT responses, without its usual structure, look closer to Llama's style, but I didn't test this directly.

Lessons learned

This project taught me a lot about the models themselves, not just how to classify them. I learned that Vicuna and Koala, even though they're built on LLaMA, were fine-tuned on ChatGPT conversations, so they partly write like GPT. That's why I left them out, since including them would have blurred the line between my classes. I also learned that llama-13b is a base model rather than a chat-tuned one, which explained why it was the easiest family to identify: its responses don't read like a polished assistant. On the modeling side, building the CNN showed me how a text CNN actually works. Each filter of width 3, 4, or 5 acts like a learned phrase detector, and max pooling keeps only the strongest match anywhere in the text. That's why the CNN handled the added prompt text better than the LSTM, which has to read everything in order. I expected the LSTM to win on text, but the simpler CNN beat it by about 9 points, and comparing training loss to validation scores is what showed me the LSTM was overfitting. I also learned how easy data leakage is to miss: since Arena sends each prompt to two models, I had to split by prompt, and the input-only results landing at chance confirmed that worked. Finally, on the practical side, setting a random seed let me reproduce identical results after my Colab session reset, and I learned to keep my Hugging Face token in Colab Secrets instead of pasting it into a public notebook.